

Introduction Générale

Dans un contexte de développement logiciel de plus en plus collaboratif et agile, la gestion rigoureuse des anomalies et des bugs représente un défi majeur pour toute équipe technique. Les outils de suivi classiques ne répondent plus toujours aux exigences de réactivité, de traçabilité et de collaboration des équipes modernes.

C'est dans ce cadre que s'inscrit notre projet de fin d'études, réalisé à l'Institut Supérieur des Études Technologiques de Sousse (ISET Sousse). Ce projet consiste à concevoir et développer une Plateforme de Ticketing Intelligente pour le Suivi des Anomalies, offrant un espace centralisé permettant aux équipes de déclarer, suivre et résoudre les tickets de bugs de façon structurée et efficace.

Ce travail a été réalisé au sein de l'agence E-2LYS, agence web multidisciplinaire spécialisée dans le développement de plateformes digitales sur mesure. La solution proposée repose sur une architecture moderne combinant Laravel (backend RESTful) et Vue.js 3 (frontend SPA), avec une gestion des accès basée sur des rôles distincts : Administrateur, Testeur et Développeur.

Ce rapport retrace l'ensemble des étapes de réalisation du projet selon une méthodologie agile Scrum, structurée en quatre sprints. Le premier chapitre, « Présentation du Cadre Général du Projet », introduira l'organisme d'accueil, l'étude de l'existant et la solution proposée. Le deuxième chapitre, « Spécification des Besoins et Choix Méthodologiques et Techniques », détaillera les acteurs, les besoins fonctionnels et non fonctionnels, la démarche Scrum adoptée, ainsi que les choix technologiques retenus. Les chapitres suivants présenteront les différents sprints réalisés, de la conception à la livraison des interfaces.

Chapitre 1 : Présentation du Cadre Général du Projet

Introduction

Ce chapitre établit le cadre général de notre projet de fin d'études. Nous présenterons d'abord l'organisme d'accueil dans sa globalité, suivi d'un diagnostic approfondi de l'existant afin d'appréhender les processus opérationnels actuels et leurs limites. Cette analyse nous permettra d'identifier précisément la problématique à résoudre et d'exposer la solution envisagée.

1. Présentation de l'organisme d'accueil

Dans le cadre de la réalisation de notre projet de fin d'études, nous avons eu l'opportunité d'effectuer notre stage au sein de l'agence **E-2LYS**. Avant de présenter le projet qui nous a été confié, il est essentiel de commencer par une présentation globale de cette entreprise, son organisation interne, ainsi que les technologies et projets qu'elle maîtrise.

1.1. Présentation générale de l'entreprise

E-2LYS est une agence web multidisciplinaire tunisienne fondée en **2011**, spécialisée dans le développement de plateformes web et l'accompagnement de la transformation digitale de ses clients. Elle intervient dans de nombreux secteurs d'activité tels que le corporate, l'industrie, l'agroalimentaire, la santé, l'éducation, le gouvernement, le tourisme et l'événementiel.

L'agence opère à travers trois sociétés et deux marques, couvrant le marché tunisien, le marché français et les marchés export africains. Son approche repose sur des compétences analytiques, créatives et techniques, offrant une prestation globale qui inclut le conseil, la stratégie digitale, le design et le développement sur mesure.

Le tableau ci-dessous présente les données essentielles concernant l'agence E-2LYS.

Dénomination	E-2LYS
Date de création	2011
Spécialité	Développement de plateformes web & transformation digitale
Marchés	Tunisie (local + export) · France · Afrique
Effectif	37 collaborateurs
Technologies principales	Laravel · Vue.js · React · WordPress · Mobile (iOS/Android)
Structure	3 sociétés — 2 marques

Tableau 1 : Fiche signalétique de l'agence E-2LYS

Chapitre 1 : Présentation du Cadre Général du Projet

1.2. Les départements de l'agence

L'agence E-2LYS est structurée autour de plusieurs pôles complémentaires :

- Pôle Développement Backend (18 développeurs) : conception, développement et maintenance de plateformes digitales en Laravel, avec intégration des API REST et gestion des bases de données.
- Pôle Développement Frontend (14 développeurs) : intégration web et mobile, développement d'interfaces utilisateur en Vue.js, React et technologie mobile.
- Pôle Design (3 designers) : conception UI/UX,
- charte graphique et expérience utilisateur.
- Pôle Gestion de Projets (3 chefs de projets) : pilotage des livrables, coordination entre les équipes et suivi de la qualité.
- Pôle Qualité (2 testeurs) : validation fonctionnelle et tests de non-régression des livrables.
- Pôle Administratif (RH & Finance) : gestion des ressources humaines, de la comptabilité et de la coordination interne.

Nous avons été accueillis au sein du Pôle Développement, où notre mission était de concevoir et développer une plateforme de ticketing pour le suivi des anomalies.

1.3. Les projets de l'agence E-2LYS

Parmi les réalisations notables de l'agence E-2LYS, on peut citer :

- Hutchinson HES : plateforme de partage des initiatives d'excellence au sein du groupe industriel Hutchinson (plus de 1000 utilisateurs worldwide).
 - Plateformes e-commerce et sites institutionnels pour des entreprises tunisiennes et françaises dans des secteurs variés.
 - Solutions de gestion interne (ERP, CRM, portails RH) développées sur mesure en Laravel.
 - Applications mobiles iOS/Android pour des clients dans les secteurs de la santé et de l'éducation.
-

Chapitre 1 : Présentation du Cadre Général du Projet

2. Étude de l'existant

L'étude de l'existant constitue une étape fondamentale dans la conduite de notre projet. Elle permet d'analyser les pratiques actuellement en place au sein des équipes de développement pour gérer les anomalies et les bugs, d'identifier leurs limites, et de dégager les besoins réels auxquels notre solution devra répondre.

2.1. Description de l'existant

Actuellement, de nombreuses équipes de développement gèrent le suivi de leurs anomalies de manière informelle et non centralisée. Les pratiques observées se résument principalement à :

- L'utilisation de fichiers tableurs (Excel, Google Sheets) pour lister et suivre les bugs signalés, sans structuration ni historique fiable.
- La communication par messagerie instantanée (WhatsApp, Telegram, Slack) ou par email pour signaler les anomalies, sans traçabilité ni centralisation des échanges.
- L'absence d'un workflow défini pour le cycle de vie des tickets : les anomalies sont signalées de façon informelle et leur résolution n'est pas systématiquement documentée.
- Le manque de visibilité pour les équipes : il n'existe pas de tableau de bord permettant de suivre l'avancement global des corrections ni d'avoir des statistiques par projet ou par membre.
- L'absence de système de notification : les membres de l'équipe ne sont pas automatiquement alertés lors de la création, la modification ou la résolution d'un ticket.

2.2. Critique de l'existant et problématique

L'observation des pratiques actuelles a mis en évidence plusieurs points forts et plusieurs insuffisances importantes.

Points forts du mode actuel :

- Simplicité de mise en place : les outils comme Excel ou la messagerie ne nécessitent aucune formation.
- Flexibilité : les équipes adaptent leurs pratiques selon leurs besoins immédiats.

Insuffisances et problèmes identifiés :

- Absence de traçabilité : il est impossible de consulter l'historique complet des modifications apportées à un ticket ou de savoir qui a fait quoi et quand.
 - Risque de perte d'informations : les échanges par messagerie sont éphémères et les fichiers Excel ne sont pas structurés pour garantir l'intégrité des données.
 - Manque de collaboration structurée : sans outil dédié, la communication entre testeurs, développeurs et administrateurs est fragmentée.
 - Absence de contrôle d'accès : n'importe quel membre peut modifier ou supprimer des données, ce qui compromet la fiabilité du suivi.
-

Chapitre 1 : Présentation du Cadre Général du Projet

- Impossibilité de produire des statistiques : il est difficile d'évaluer la performance de l'équipe, le taux de résolution ou les tendances par projet.

Ces constats soulèvent une question centrale : comment centraliser, structurer et sécuriser le suivi des anomalies tout en permettant une collaboration fluide entre les différents acteurs de l'équipe ?

3. Proposition d'une solution

Face aux limites identifiées, la solution que nous proposons consiste à développer une Plateforme de Ticketing Intelligente pour le Suivi des Anomalies, une application web moderne qui centralise et structure l'ensemble du processus de déclaration, suivi et résolution des bugs au sein d'une équipe de développement.

La plateforme a pour ambition de :

- Offrir un système d'authentification sécurisé avec gestion des rôles (Administrateur, Testeur, Développeur), assurant un contrôle d'accès strict à chaque fonctionnalité.
- Permettre aux testeurs de créer des tickets structurés (titre, description, priorité, projet) et de suivre leur avancement en temps réel.
- Donner aux développeurs un espace dédié pour consulter les tickets qui leur sont assignés, mettre à jour leur état et documenter leur résolution via des commentaires.
- Fournir à l'administrateur les outils nécessaires pour gérer les utilisateurs, les projets, les affectations et accéder à un tableau de bord statistique.
- Garantir la traçabilité complète : chaque modification de ticket est horodatée et attribuée à son auteur, sans possibilité de suppression.
- Automatiser les notifications par email lors des événements clés : validation de compte, changement d'état d'un ticket, fermeture d'un projet.

La mise en place de cette plateforme apportera plusieurs bénéfices concrets. Elle permettra d'éliminer les risques de perte d'information, d'améliorer la réactivité de l'équipe face aux anomalies, et de fournir des indicateurs de performance clairs pour piloter la qualité du développement. L'architecture modulaire de la solution permettra également d'intégrer facilement de nouvelles fonctionnalités à l'avenir, comme un module d'intelligence artificielle pour la classification automatique des tickets.

Conclusion

Après avoir établi le cadre général du projet et identifié la problématique organisationnelle à résoudre, le chapitre suivant présentera les spécifications fonctionnelles et techniques nécessaires au développement de la plateforme, ainsi que la méthodologie agile adoptée pour structurer notre travail.

Chapitre 1 : Présentation du Cadre Général du Projet

Introduction

La phase de spécification et d'analyse des besoins est une étape fondamentale dans la conduite d'un projet logiciel. Elle permet de structurer le système en identifiant ses acteurs clés, en formalisant leurs besoins et en définissant les choix méthodologiques et technologiques qui guideront l'ensemble du développement.

Dans ce chapitre, nous détaillons, dans un premier temps, les exigences fonctionnelles et non fonctionnelles auxquelles la plateforme devra répondre. Ensuite, nous exposons le cadre méthodologique Scrum adopté, garantissant une gestion itérative et collaborative du projet. Enfin, nous justifions les choix technologiques retenus pour la réalisation de cette solution.

1. Identification des acteurs

Un acteur représente un rôle assumé par une entité externe (humaine ou système) interagissant avec la plateforme. Dans notre système de ticketing, trois acteurs principaux ont été identifiés :

Acteur	Description
Administrateur	L'acteur central disposant des droits les plus étendus. Il supervise la plateforme, gère les comptes utilisateurs (validation, désactivation), crée et gère les projets, affecte les membres, consulte tous les tickets et accède au tableau de bord statistique.
Testeur	Membre de l'équipe responsable de la détection et du signalement des anomalies. Il crée les tickets de bugs, suit leur avancement, ajoute des commentaires et ferme les tickets dont la résolution a été confirmée.
Développeur	Membre de l'équipe chargé de la résolution des anomalies. Il consulte les tickets des projets qui lui sont assignés, met à jour leur état (En cours → Résolu) et documente techniquement les corrections apportées.

Tableau 2 : Identification des acteurs du système

2. Les besoins fonctionnels

Dans cette partie, nous présentons en détail les fonctionnalités associées à chaque acteur de la plateforme. Cette description couvre l'ensemble du périmètre fonctionnel, depuis la gestion des utilisateurs jusqu'au tableau de bord de supervision.

Chapitre 2 : Spécification des Besoins et Choix Méthodologique et Techniques

2.1. Fonctionnalités liées à l'Administrateur

L'application doit permettre à l'administrateur authentifié de :

- Consulter la liste complète de tous les utilisateurs inscrits sur la plateforme.
- Valider un compte en attente et déclencher automatiquement l'envoi d'un email de notification au membre accepté.
- Rejeter une demande d'inscription et envoyer automatiquement un email d'information au membre concerné.
- Désactiver un compte actif sans le supprimer, afin de préserver la traçabilité des données.
- Gérer son compte personnel : modifier son mot de passe et son adresse email.
- Créer des projets en définissant leur nom, description et statut initial.
- Modifier les informations d'un projet existant et le fermer lorsque le travail est terminé.
- Affecter des membres (testeurs et développeurs) à un projet et les retirer si nécessaire.
- Filtrer et rechercher des projets par nom pour retrouver rapidement l'information.
- Consulter tous les tickets de la plateforme, suivre leur état et les fermer si nécessaire.
- Accéder au tableau de bord avec des statistiques détaillées par projet, par développeur et par testeur.

2.2. Fonctionnalités liées au Testeur

L'application doit permettre au testeur authentifié de :

- Créer un compte et accéder à la plateforme après validation par l'administrateur.
- S'authentifier et gérer son compte (changer mot de passe ou adresse email).
- Consulter uniquement les projets auxquels il est affecté et les filtrer par nom.
- Créer un ticket d'anomalie en précisant le titre, la description, la priorité et le projet concerné.
- Modifier les informations de ses propres tickets (description, priorité) avant qu'ils soient pris en charge.
- Suivre l'état d'avancement de ses tickets en temps réel.
- Ajouter des commentaires sur un ticket pour communiquer avec le développeur ou fournir des informations complémentaires.
- Fermer un ticket lorsqu'il confirme que l'anomalie est définitivement résolue.

2.3. Fonctionnalités liées au Développeur

L'application doit permettre au développeur authentifié de :

- Créer un compte et accéder à la plateforme après validation par l'administrateur.
 - S'authentifier et gérer son compte (changer mot de passe ou adresse email).
 - Consulter les projets qui lui sont assignés et les tickets associés.
 - Mettre à jour l'état d'un ticket : passer un ticket de 'Ouvert' à 'En cours', puis à 'Résolu'.
-

Chapitre 2 : Spécification des Besoins et Choix Méthodologique et Techniques

- Ajouter des commentaires techniques pour documenter la résolution et communiquer avec le testeur.

3. Les besoins non fonctionnels

Outre les fonctionnalités métier, la plateforme devra satisfaire un ensemble d'exigences de qualité :

Critère	Exigence
Sécurité	Authentification JWT sécurisée, hashage bcrypt des mots de passe, protection contre les injections SQL et attaques CSRF.
Ergonomie	Interface claire, intuitive et responsive, adaptée aux écrans desktop et mobile, avec des messages d'erreur explicites en français.
Maintenabilité	Code structuré selon le pattern MVC (Laravel), API REST documentée, séparation claire entre backend et frontend.
Scalabilité	Architecture modulaire permettant l'ajout futur de fonctionnalités (ex: module IA pour la classification automatique des tickets).
Traçabilité	Historique complet et horodaté de tous les changements effectués sur les tickets. Aucune suppression de données n'est autorisée.
Performance	Temps de réponse des API inférieur à 2 secondes pour les opérations courantes.

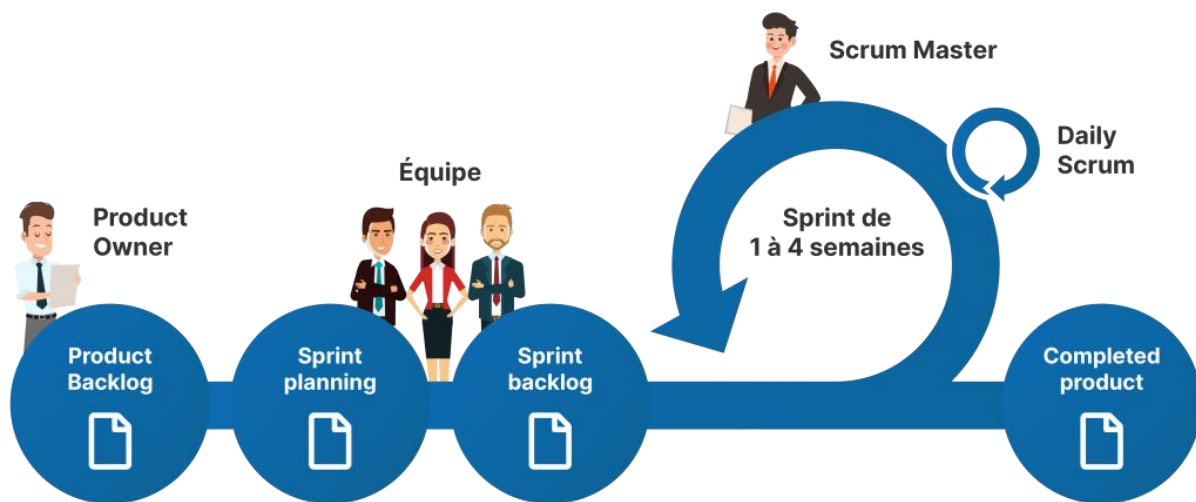
Tableau 3 : Besoins non fonctionnels

4. Le Cadre Méthodologique Scrum

4.1. Présentation de Scrum

Pour ce projet, nous avons adopté la méthodologie Scrum, un cadre agile permettant de livrer des fonctionnalités de manière incrémentale à travers des itérations courtes appelées Sprints. Cette approche favorise la collaboration, l'adaptabilité et la livraison continue de valeur.

Le cycle Scrum commence par le Product Backlog, liste priorisée des fonctionnalités souhaitées, puis passe par le Sprint Planning où l'équipe sélectionne les tâches pour constituer le Sprint Backlog. Le cœur du processus est le Sprint (2 semaines dans notre cas), pendant lequel l'équipe développe les fonctionnalités planifiées. Chaque sprint se clôture par une Sprint Review (démonstration des livrables) et une Sprint Retrospective (bilan et amélioration continue).



4.2. Les Rôles Scrum

Le framework Scrum définit des rôles précis permettant à chaque membre d'assumer des responsabilités claires et complémentaires :

- **Product Owner (PO)** : responsable de la définition et de la priorisation des besoins fonctionnels. Il est représenté par l'encadrante académique du projet.
- **Scrum Master (SM)** : garant de l'application correcte de Scrum. Le rôle est assuré en alternance par Maha Dorai et Ahmed Skhiri.
- **Équipe de Développement** : groupe auto-organisé responsable de la réalisation technique. Composée de Maha Dorai (frontend & intégration) et Ahmed Skhiri (backend & API).

Rôle	Mission	Acteur(s)
Product Owner	Définit les besoins fonctionnels et priorise le backlog	Encadrante académique
Scrum Master	Veille à l'application correcte de Scrum	Maha Dorai / Ahmed Skhiri (alternance)
Équipe de développement	Réalise le travail technique (backend + frontend)	Maha Dorai & Ahmed Skhiri

Tableau 4 : Répartition des rôles Scrum

5. Le Backlog du Produit

Le backlog produit regroupe l'ensemble des besoins fonctionnels exprimés sous forme de user stories. Chaque entrée comprend un identifiant unique, un thème, une description claire du besoin et un niveau de priorité défini selon la méthode MoSCoW :

- **M (Must have)** : indispensables au fonctionnement de la plateforme.
- **S (Should have)** : importantes mais pouvant être reportées à un sprint ultérieur.

Chapitre 2 : Spécification des Besoins et Choix Méthodologique et Techniques

- C (Could have) : optionnelles selon les ressources disponibles.

ID	Thème	User Story	Priorité	Sprint
US01	Inscription	En tant que membre, je veux créer un compte avec email et mot de passe, afin d'accéder à la plateforme après validation de l'administrateur.	Must	S1
US02	Validation compte	En tant qu'admin, je veux valider ou rejeter un compte en attente, afin de contrôler les accès à la plateforme (avec envoi d'email de notification).	Must	S1
US03	Authentification	En tant qu'utilisateur, je veux m'authentifier avec mes identifiants, afin d'accéder à mon espace personnel et aux fonctionnalités correspondant à mon rôle.	Must	S1
US04	Réinit. mdp	En tant qu'utilisateur, je veux recevoir un lien de réinitialisation par email, afin de récupérer l'accès à mon compte en cas d'oubli du mot de passe.	Should	S1
US05	Créer projet	En tant qu'admin, je veux créer et gérer des projets (créer, modifier, fermer), afin d'organiser le travail autour d'un périmètre fonctionnel défini.	Must	S2
US06	Affecter membres	En tant qu'admin, je veux affecter des testeurs et des développeurs à un projet, afin que chaque membre n'accède qu'aux projets qui le concernent.	Must	S2
US07	Consulter projets	En tant que membre, je veux consulter uniquement les projets auxquels je suis affecté, avec la possibilité de filtrer et rechercher par nom.	Must	S2
US08	Créer ticket	En tant que testeur, je veux créer un ticket d'anomalie (titre, description, priorité, projet), afin de signaler un bug à l'équipe de développement.	Must	S3
US09	Modifier état	En tant que développeur, je veux modifier l'état d'un ticket (En cours / Résolu), afin de communiquer l'avancement de la correction à l'équipe.	Must	S3
US10	Fermer ticket	En tant que testeur ou admin, je veux fermer un ticket dont le bug est confirmé résolu, afin de clôturer définitivement le suivi de l'anomalie.	Must	S3

Chapitre 2 : Spécification des Besoins et Choix Méthodologique et Techniques

US11	Commentaires	En tant qu'utilisateur, je veux ajouter des commentaires sur un ticket, afin de documenter l'avancement et faciliter la communication entre les membres.	Should	S3
US12	Tableau bord	En tant qu'admin, je veux accéder à un tableau de bord avec des statistiques par projet, par développeur et par testeur, afin de piloter la qualité du développement.	Should	S4
US13	Filtres avancés	En tant qu'utilisateur, je veux filtrer et rechercher des tickets selon plusieurs critères (état, priorité, projet), afin de retrouver rapidement les anomalies qui me concernent.	Could	S4

Tableau 5 : Backlog Produit Global

6. Planification des Sprints

La planification des sprints mobilise l'ensemble de l'équipe Scrum. En définissant collectivement les objectifs et en sélectionnant les éléments prioritaires du backlog, nous garantissons des estimations réalistes et un engagement partagé, facteurs déterminants pour la réussite de chaque cycle de développement.

Sprint	Durée	Objectif	Livrables
Sprint 1	2 sem.	Authentification & Gestion des utilisateurs	Inscription · Login JWT · Validation compte · Reset mot de passe · Système de mailing
Sprint 2	2 sem.	Gestion des projets	CRUD projets · Affectation membres · Filtrage · Consultation par rôle
Sprint 3	2 sem.	Gestion des tickets	Création · Modification · Changement d'état · Fermeture · Commentaires · Notifications
Sprint 4	2 sem.	Tableau de bord & Finalisation	Statistiques · Tests finaux · Déploiement · Documentation

Tableau 6 : Feuille de Route des Sprints (Roadmap Agile)

7. Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation constitue un outil visuel essentiel pour représenter les interactions entre les différents acteurs et le système développé. Il offre une vue d'ensemble synthétique des fonctionnalités principales de la plateforme, tout en identifiant les utilisateurs concernés.

Chapitre 2 : Spécification des Besoins et Choix Méthodologique et Techniques

En raison de la complexité du diagramme global, nous avons choisi de le subdiviser par package d'acteurs, afin de mieux structurer l'information et de clarifier les interactions spécifiques de chaque profil avec le système.

7.1. Diagramme de cas d'utilisation : Package Administrateur

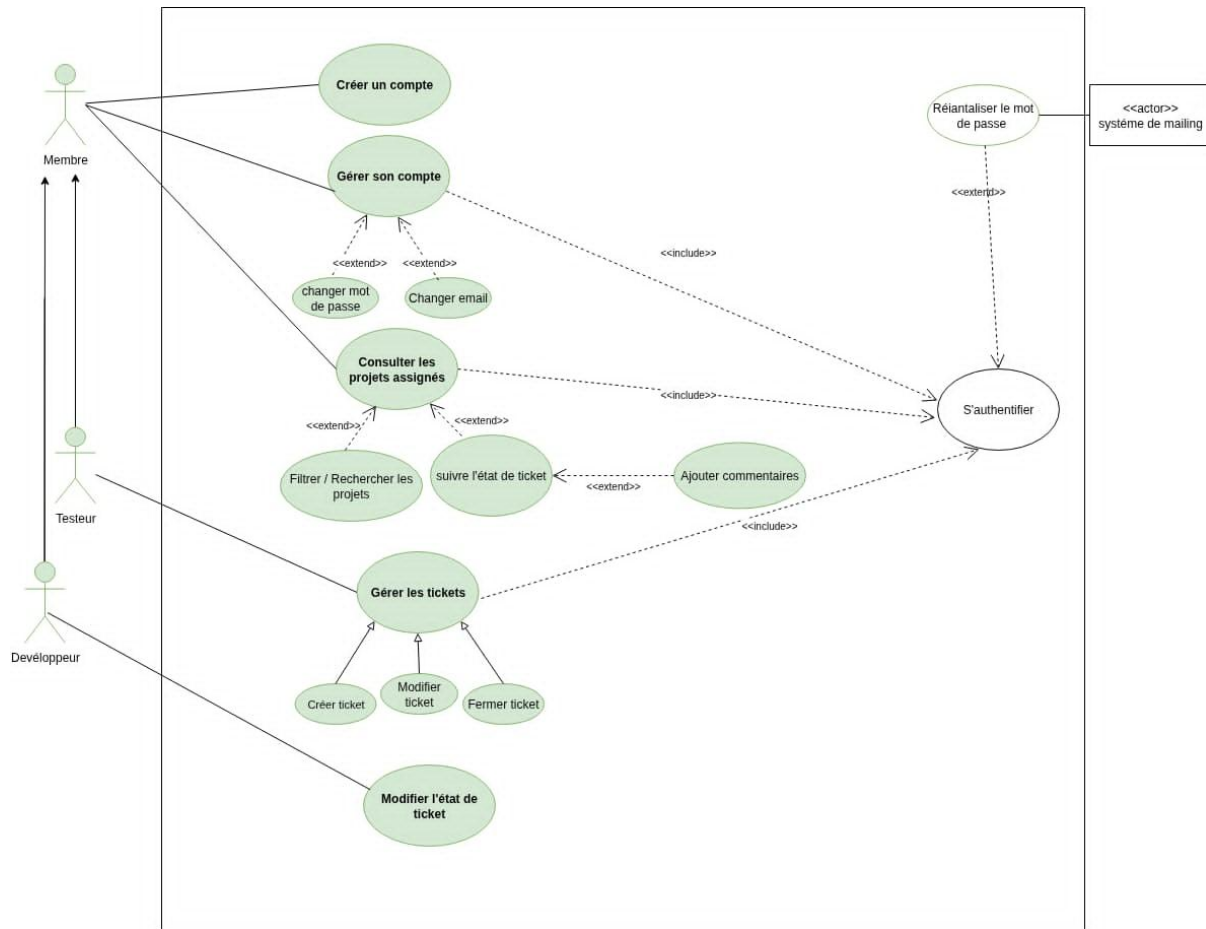
La figure suivante illustre le diagramme de cas d'utilisation du côté administrateur. Cet acteur dispose des droits les plus étendus sur la plateforme : il gère les utilisateurs, les projets, les tickets et accède au tableau de bord statistique.



7.2. Diagramme de cas d'utilisation : Package Membre (Testeur / Développeur)

La figure suivante illustre le diagramme de cas d'utilisation du côté membre. Le testeur peut créer, modifier et fermer des tickets ; le développeur peut mettre à jour l'état des tickets et ajouter des commentaires. Les deux partagent les fonctionnalités de gestion de compte et de consultation des projets assignés.

Chapitre 2 : Spécification des Besoins et Choix Méthodologique et Techniques



8. Choix Techniques

Cette section présente l'ensemble des décisions techniques qui ont guidé la réalisation du projet, depuis le choix du framework de développement jusqu'aux outils de conception et de versioning.

8.1. Outils de conception et de développement

Outil	Description et justification
Visual Studio Code	Éditeur de code principal utilisé pour le développement frontend (Vue.js) et backend (Laravel). Son large écosystème d'extensions (Volar pour Vue, PHP Intelephense), son débogueur intégré et son terminal intégré en font l'outil idéal pour ce projet.
draw.io	Outil de modélisation utilisé pour la création de tous les diagrammes UML : diagrammes de cas d'utilisation, de séquence et de classes. Sa prise en main intuitive et son export en haute définition facilitent l'intégration dans le rapport.
GitHub	Plateforme de versioning collaborative utilisée pour la gestion du code source. Elle permet la synchronisation du travail entre les deux

Chapitre 2 : Spécification des Besoins et Choix Méthodologique et Techniques

	membres de l'équipe, le suivi des modifications et la sauvegarde du code.
Postman	Outil de test des API REST utilisé pour valider les endpoints du backend Laravel avant leur intégration dans le frontend Vue.js.
MySQL Workbench	Outil graphique de gestion de la base de données MySQL utilisé pour la conception du schéma relationnel et la visualisation des données.

Tableau 7 : Outils de développement

8.2. Framework de développement

Pour le développement du backend, nous avons opté pour le framework Laravel, un framework web open source créé par Taylor Otwell en 2011, écrit en PHP et basé sur le pattern MVC (Modèle-Vue-Contrôleur). Laravel est reconnu pour sa rapidité, sa robustesse et son évolutivité. Il intègre nativement des fonctionnalités essentielles telles que l'authentification JWT (via le package tymon/jwt-auth), la gestion des bases de données via l'ORM Éloquent, le routage RESTful et le système de migrations.

Pour le frontend, nous avons choisi Vue.js 3, un framework JavaScript progressif basé sur le pattern MVVM. Vue.js 3 offre une réactivité performante, une organisation modulaire via les composants, et une excellente intégration avec les API REST grâce à la bibliothèque Axios. La gestion de l'état de l'application est assurée par Pinia, le gestionnaire d'état officiel de Vue.js 3.

8.3. Architecture de la solution

8.3.1. Pattern MVC (Backend Laravel)

L'architecture de notre solution repose sur le pattern MVC (Modèle-Vue-Contrôleur), qui sépare l'application en trois couches distinctes :

- Le Modèle (Model) : représente les données et la logique métier. Dans Laravel, chaque modèle Éloquent correspond à une table de la base de données et encapsule toutes les interactions avec celle-ci.
- La Vue (View) : dans notre architecture SPA, cette couche est entièrement gérée par Vue.js 3 côté frontend. Elle affiche les données reçues via les API REST.
- Le Contrôleur (Controller) : traite les requêtes HTTP entrantes, interagit avec les modèles et retourne des réponses JSON structurées au frontend.

[Insérer l'image ici]

Figure 4 : Architecture MVC de la solution

Chapitre 2 : Spécification des Besoins et Choix Méthodologique et Techniques

8.3.2. Architecture RESTful API

La communication entre le frontend Vue.js et le backend Laravel est assurée par une API RESTful sécurisée via JSON Web Tokens (JWT). Chaque requête du frontend est accompagnée d'un token JWT dans le header d'autorisation. Le backend valide ce token, identifie le rôle de l'utilisateur et renvoie uniquement les données auxquelles il est autorisé à accéder.

8.3.3. Gestion de la base de données

Nous avons opté pour **MySQL** comme système de gestion de base de données relationnelle. La gestion des migrations et des relations entre les tables est entièrement assurée par l'ORM Éloquent de Laravel, qui abstrait les requêtes SQL et garantit l'intégrité référentielle des données.

9. Diagramme de GANTT

Le diagramme de Gantt permet de visualiser les différentes tâches du projet, leur durée, leurs dates de début et de fin, ainsi que les relations de dépendance entre elles. Il offre une vue claire de l'avancement global et des jalons importants du projet.



Figure 5 : Diagramme de GANTT du projet

Le projet s'étale sur une période de 8 semaines, structurée en quatre sprints successifs de deux semaines chacun. La phase de lancement (analyse des besoins, conception de l'architecture et mise en place de l'environnement de développement) précède le premier sprint. La rédaction du rapport progresse en parallèle avec le développement tout au long du projet.

Conclusion

Ce chapitre nous a permis de définir avec précision les besoins fonctionnels et non fonctionnels de la plateforme, d'établir le cadre méthodologique Scrum qui structurera notre travail, et de justifier les choix technologiques retenus. Les chapitres suivants présenteront le détail de chaque sprint réalisé, de la conception UML à la livraison des interfaces.